Suggested Teaching Guidelines for

Algorithms and Data Structures using Java

PGCP-AC, August 2026

Duration: 90 Hours: 45 Theory (T) + 45 Lab (L)

Objective: To equip students with problem-solving and computational thinking skills by applying fundamental and advanced data structures, algorithms, and algorithm design techniques to develop efficient and scalable software solutions.

Prerequisites: Knowledge of Java Programming

Evaluation method: Theory exam – 40%, Lab exam – 40%, Internal Assessment – 20%

List of Course Material

Text book:

1. Object-Oriented Programming and Data Structures using Java, 2026 edition, Cay S. Horstmann / Wiley

Reference book:

1. Data Structures and Algorithms in Java, 6th edition, Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, Subhasish Banerjee / Wiley
2. Algorithms, 4th Edition, Robert Sedgewick, Kevin Wayne / O'Reilly

Session 1: Problem Solving & Computational Thinking (2T + 2L)

- Define the problem
- Identify the problem
- Introduction to Problem Solving
- Problem solving basics

Lab:

- Faculty needs to assign different problems, mostly real-world problems
- Students need to analyze as per the techniques learned
- They need to implement the selected solution and need to do the documentations.
- Introducing the mini project ideas

Sessions 2 & 3: Algorithms & Data Structures (4T + 4L)

- Introductory Concepts
- Algorithm Constructs
- Complexity analysis of algorithms (Big O notation)
- OO design: Abstract Data Types (ADTs)
- Basic Data Structures
  - Arrays
  - Stacks
  - Queues
  - Circular Queues

Suggested Teaching Guidelines for

Algorithms and Data Structures using Java

PGCP-AC, August 2026

Lab:

- Implement stack through array
- Complexity analysis of loops and recursive algorithms.
- Implement queues with inserting element at different location (First, Last)
- Implement circular queue

Sessions 4 & 5: Linked List Data Structures (4T + 4L)

- Linked List
  - Singly linked lists
  - Doubly linked lists
  - Circular linked lists
  - Node-based storage with arrays

Lab:

- Implement circular queue using linked list
- Implement stack using linked list

Session 6: Recursion (2T + 2L)

- Introduction to Recursion
- Base condition in recursion.
- Direct and indirect recursion.
- Memory is allocated to different function calls in recursion.
- Pro and cons of recursion
- Function complexity during recursion

Sessions 7, 8 & 9: Trees & Applications (6T + 6L)

- Introduction to trees
- Trees and terminology
- Tree traversals
- Binary trees
- Complete binary trees / Almost complete binary tree (ACBT)
- Array implementation of ACBT
- Binary search trees
- AVL tree

Lab:

- Write a program to implement a binary search tree and the following operations on it:
  - Create()
  - Tree traversals- Breadth First Search, Depth First Search, Inorder(), Preorder(), Postorder()
  - Delete()

Suggested Teaching Guidelines for

Algorithms and Data Structures using Java

PGCP-AC, August 2026

Sessions 10, 11 & 12: Searching and Sorting Algorithms (6T + 6L)

- Objectives of Searching
  - The Sequential Search
  - Analysis of Sequential Search
  - The Binary Search
- Analysis of Binary Search
- Introduction to sorting
  - Selection sort
  - Insertion sort
  - Bubble sort
  - Heapsort
  - Merge sort
  - Quicksort

Lab:

- Writing program to search an item through sequential search technique.
- Implement to find an item in a list through binary search
- Implement sorting algorithm for: insertion sort, Quicksort

Session 13: Hash Functions and Hash Tables (2T + 2L)

- Hashing & Introduction to hash tables
- Hash Functions
- Different type of hash functions
- Collision Resolution
- Linear Probing
- Quadratic Probing
- Double Hashing
- Inserting and deleting an element from a hash table

Lab:

- Implement hashing techniques in different programs solved earlier
- Write a program to implement Hash table
- Fibonacci recursive algorithm improvement using hash table

Suggested Teaching Guidelines for

Algorithms and Data Structures using Java

PGCP-AC, August 2026

Sessions 14, 15, 16 & 17: Graphs and Applications (8T + 8L)

- Introduction to graph theory
- Graph Terminology
- Different types of Graphs
- Representation of Graphs
  - Adjacency Matrix
  - Adjacency List
  - Graph Traversal Algorithms (Breadth First Search, Depth First Search)
- Shortest Path
  - Level Setting: Dijkstra's algorithm
  - Level Correcting: All-pairs shortest path, Floyd-Warshall algorithm
- Spanning Trees
  - Minimum spanning tree algorithms,
  - Prim's algorithm
  - Kruskal's Algorithm

Lab:

- Implement a graph using adjacency Matrix and traverse using Depth First Search.
- Implement a graph and do traversal using stack and queue.

Sessions 18, 19, 20, 21 & 22: (11T+ 11L)

- Introduction to algorithm design techniques
- Writing efficient Algorithms
- Algorithm Design techniques
- Analysis of an Algorithm
  - Asymptotic Analysis
  - Algorithm Analysis
- Analysis of different type of Algorithms
  - Divide and Conquer Algorithm
  - Greedy algorithm
  - Dynamic Programming algorithm
  - Brute force algorithm
  - Backtracking algorithms
  - Branch-and-bound algorithms
  - Stochastic algorithms
- Complexity
  - Complexity Analysis
  - Space complexity of algorithm
  - Time complexity of algorithm
- Case study on Algorithm Design techniques
- Application of Data structures

Suggested Teaching Guidelines for

Algorithms and Data Structures using Java

PGCP-AC, August 2026

Lab:

- Find the most efficient algorithm to search for an element in a large array and compare its time complexity with linear search
- Implement Merge Sort and Quick Sort, then compare their execution time for different input sizes
- Solve the Fractional Knapsack problem using a greedy approach and analyze its time complexity
- Implement the 0/1 Knapsack problem using dynamic programming and compare it with a recursive solution
- Solve the N-Queens problem using backtracking and display all possible solutions for a given board size
- Design and analyze an algorithm to find the shortest path between two nodes in a weighted graph using an appropriate algorithm
- Compare the time and space complexity of different sorting and searching algorithms, and identify the most suitable algorithm for various input scenarios